# Event Management System

SECP3623 Database Programming | Section 1

| Name | Matric No |
| --- | --- |
| Brendan Chia Yan Fei | A23CS0211 |
| Sabrina Heng Wei Qi | A23CS0265 |
| Lau Yan Kai | A23CS0098 |
| Gui Kah Sin | A23CS0080 |

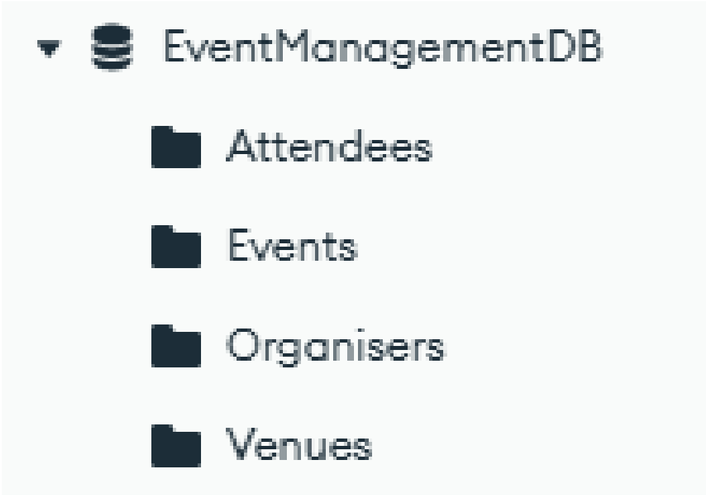
# Event Management System

Traditional databases often struggle with event schedules and the different types of information found in technology workshops

By utilizing a document-oriented model, the system is designed to handle complex relationships between events, venues, organisers, and attendees with high scalability and flexibility.

Core Functionality: Handles data storage for registration tracking and ticket inventory.

# Event Management Schema



# Collections

## 1.Attendees

```
_id: ObjectId('695cd174a392d4601b884d9d')
name : "Sarah Tan"
email : "sarah.tan@techcorp.com"
type : "Professional"
registered_events : Array (2)
    0: ObjectId('695cc388fc9b8ea1d1024a16')
    1: ObjectId('695cca0aa392d4601b884d96')
interests : Array (2)
    0: "Data Engineering"
    1: "Fintech"
```

## 2.Events

```
_id: ObjectId('695cc07afc9b8ea1d1024a15')
title : "Cloud-Native Development Workshop"
category : "Technology"
date : 2026-03-15T09:00:00.000+00:00
venue_id : ObjectId('6598ef01c2b5a12345678901')
organizer_id : ObjectId('6598f122c2b5a12345678910')
description : "Hands-on session regarding Azure and MongoDB Atlas integration."
schedule : Array (3)
    0: Object
        time : "09:00 AM"
        activity : "Introduction to NoSQL"
    1: Object
    2: Object
tickets : Array (2)
    0: Object
        type : "Student"
        price : 25
        available_seats : 30
    1: Object
tags : Array (4)
    0: "Cloud"
    1: "Database"
    2: "UTM"
    3: "Workshop"
```

# Collections

## 3.Organisers

```
_id: ObjectId('6598f122c2b5a12345678914')
org_name : "SEA Finance Hub"
contact_email : "events@seafinance.com"
website : "https://seafinance.com"
rating : 4.7
specialization : Array (3)
  0: "Fintech"
  1: "Blockchain"
  2: "Digital Banking"
verified_status : true
```

established_year
location
social_media

## 4.Venues

```
_id: ObjectId('6598ef01c2b5a12345678902')
name : "Persada Johor International Convention Centre"
address : "Jalan Abdullah Ibrahim, 80000 Johor Bahru"
capacity : 3000
facilities : Array (4)
  0: "Exhibition Hall"
  1: "Catering Service"
  2: "Parking"
  3: "Video Conferencing"
contact_number : "+60 7-219 8888"
```

# Schema Design Strategy

## 1.Embedding (Embedded Array)

- Ticket tiers and event schedules within the **Events** collection.
- Always retrieved together with event details, reducing database read operations.

## 2.Referencing

- Linking the Events collection to Organisers and Venues via manual ObjectId references.

```
_id: ObjectId('695cc77ca392d4601b884d95')
title : "Global AI & Robotics Summit 2026"
category : "Technology"
date : 2026-05-05T09:00:00.000+00:00
venue_id : ObjectId('6598ef01c2b5a12345678901')
organizer_id : ObjectId('6598f122c2b5a12345678913')
```

# BSON Implementation and Data Types

- ObjectId: Used for primary keys (_id) and manual references to link events to their organizers
- Embedded Objects/Arrays:

1. Schedule: Array of embedded objects containing time and activity.
2. Tickets: Array of objects for different ticket tiers (Student vs. Professional).
3. Tags: Simple array of strings for efficient event categorization and searching

- Date & Numbers: Used for temporal fields (start times) and numeric values (ticket prices, seat capacity) to enable efficient filtering and sorting

# Insert operations

## `insertOne()`

to add individual records

```
db.Attendees.insertOne({
  name: "Poh Lok Yee",
  email: "lokyee@gmail.com",
  type: "Student",
  registered_events: [
    ObjectId("695cc07afc9b8ea1d1024a15")
  ],
  interests: ["Cloud","DevOps"]
})
```

## `insertMany()`

to insert multiple event records at once

```
db.Events.insertMany([
   {
     title: "AI Bootcamp 2026",
     category: "Techlonogy",
     date: ISODate("2026-06-01T09:00:00Z"),
     venue_id: ObjectId("6598ef01c2b5a12345678901"),
     organizer_id: ObjectId("6598f122c2b5a12345678913"),
     description: "Join us, to learn more on AI in future!",

     schedule: [
       {time: "08:30 AM", activity: "Regitration" },
       {time: "10.00 AM", activity: "Keynote: How AI changes future ?"},
       {time: "11.00 AM", activity: "Hands on session" },
       {time: "12.30 PM", activity: "Lunch Time" },
       {time: "2.00 PM", activity: "Photo session and Goodbye !"}
     ],

     tickets: [
       {type: "Student", price: 10, available_seats: 100},
       {type: "Non-Student", price: 25, available_seats: 100}
     ],
     tags: ["AI","ML","Workshop"]
   },
   {
```

# Query operations

## `find () & findOne()`

to retrieve data from collections

```
> db.Attendees.find({type: "General"})
```

```
> db.Attendees.findOne({type: "General"})
```

## `advanced filtering operators ( $eq, $in, $and, $or)`

to retrieve specific data

```
> db.Organisers.find({ org_name: { $eq: "UTM Faculty of Computing"}})
```

```
> db.Events.find({ date: { $lt: ISODate("2026-07-01T00:00:00Z") } })
```

```
> db.Venues.find({ capacity: { $gt: 2000 } })
```

```
> db.Events.find({ tags: { $in: ["Blockchain"] } })
```

```
> db.Events.find({
    $and: [
      { category: "Technology" },
      { date: { $gt: ISODate("2026-01-01T00:00:00Z") } }
    ]
  })
```

```
> db.Attendees.find({
    $or: [
      { type: "Student" },
      { type: "Academic" }
    ]
  })
< {
```

# Query operations

## Array queries

to query embedded arrays

```
> db.Venues.find({ facilities: "High-speed WiFi" })
```

---

## Projection

to return only required fields

```
> db.Attendees.find(
    {},
    { name: 1, email: 1, _id: 0 }
  )
< {
```

# Update operations

## `updateOne()`

to modify existing documents

```
> db.Attendees.updateOne(
    { email: "siti.a@utm.edu.my" },
    { $set: { type: "Professional" } }
  )
```

## `updateMany()`

```
> db.Organisers.updateMany(
    { verified_status: false },
    { $set: { verified_status: true } }
  );
```

# Update operations

## $set

to update specific field

```
> db.Events.updateOne(
    { title: "AI Bootcamp 2026" },
    { $set: { category: "Education" } }
  )
```

## $inc

to increment numeric value

```
> db.Organisers.updateOne(
    { org_name: "SEA Finance Hub" },
    { $inc: { rating: 0.2 } }
  )
```

## $push and $pull

to manage array fields

```
> db.Attendees.updateOne(
    { name: "Kevin Wong" },
    { $push: { interests: "Startups" } }
  )
```

```
> db.Attendees.updateOne(
    { name: "Kevin Wong" },
    { $pull: { interests: "Blockchain" } }
  )
```

# Delete operations

## $deleteOne()

to remove a single record

```
> db.Attendees.deleteOne({ email: "siti.a@utm.edu.my" }
```

## $deleteMany()

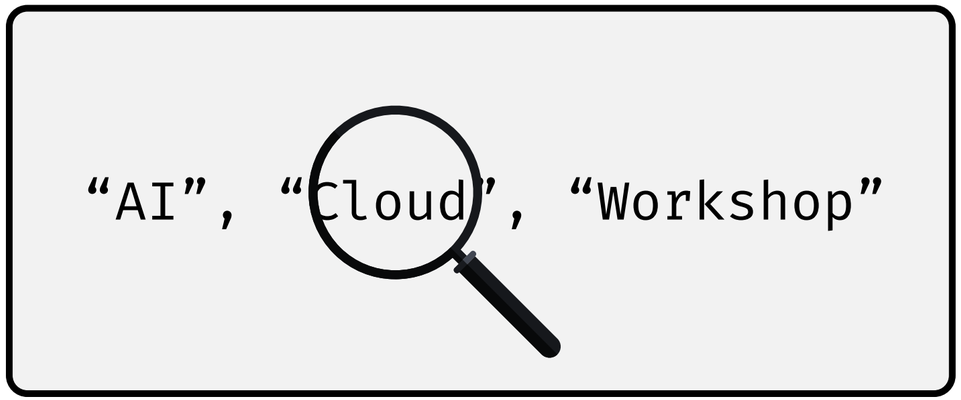
to remove a multiple records at the same time

```
> db.Events.deleteMany({ category: "Finance" })
```

# Indexing

## Single-field Index

```
createIndex({tags: 1})
```




instant keyword lookup

## Compound Index

```
createIndex({category:1,price:-1})
```

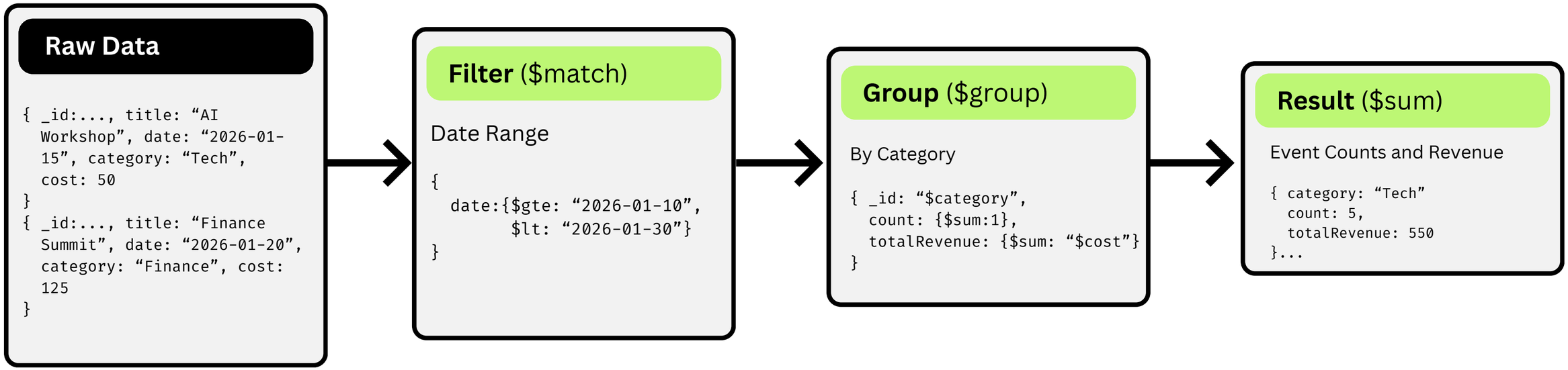
```
Technology
  [{“item”: “item A”, “price”: 20},
   {“item”: “item B”, “price”: 10},
   {“item”: “item C”, “price”: 5}
  ]
```

filter by category → sort by price

# Performance Analysis

| | Before Indexing | After Indexing |
|---|---|---|
| **Stage** | COLLSCAN (Collection Scan) | IXSCAN (Index Scan) |
| **Complexity** | O(N) - Linear | O(log N) - Logarithmic |
| **Docs Examined** | All documents | Matching document |
| **Speed** | Slow  | Instant  |

# Aggregation Pipelines




```
> db.Events.aggregate([
  {$match: {date: {$gte: “2026-01-10”, $lt: “2026-01-30”}}},
  {$group: {_id: “$category”,
            count: {$sum:1},
            totalRevenue: {$sum: “$cost”}}},
  {$sort: {totalRevenue:-1}}
  ])
```

**Event Counts and Revenue**

Process: Filter by year, group by category, count events, sum costs and sort by highest revenue.

# Team Member Contributions

Project completed with good collaboration and clear task distribution

| Team Member | Role & Contributions |
| --- | --- |
| Brendan Chia Yan Fei | • Designed NoSQL database schema<br>• Created core collections (Events, Attendees, Organisers, Venues) |
| Sabrina Heng Wei Qi | • Implemented CRUD operations<br>• Developed queries and tested outputs |
| Lau Yan Kai | • Implemented indexing and aggregation<br>• Handled sorting and performance analysis |
| Gui Kah Sin | • Analysed security and system limitations<br>• Prepared teamwork & conclusion sections<br>• Reviewed final report and code |

*Thank you*